

Smart Tourism with Multi agent collaboration Software Architecture Document

Version 1.0

Index No	Contribution
230247T	Modeled the high-level system structure by creating the Use-Case Diagram, Class Diagram, Deployment Diagram, Package Diagram, and Database ER Diagram, along with writing their detailed textual descriptions. Documented initial design tool references and defined the system's core quality attributes, including scalability, maintainability, reliability, and usability.
230279R	Authored the introduction, architectural goals, constraints, and representation details. Detailed all 17 individual use-case specification tables and modeled the dynamic system behavior across 12 Activity and Sequence Diagrams for key workflows. Defined the Implementation Layer descriptions, system size and performance requirements, and compiled the formal reference list.

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

Revision History

Date	Version	Description	Author
08/08/26	1.0	Initial release of the Software Architecture Document (SAD)	Group 08

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

Table of Contents

1.	Introduction	5
1.1	Purpose	5
1.2	Scope	5
1.3	Definitions, Acronyms, and Abbreviations	5
1.4	References	5
1.5	Overview	6
2.	Architectural Representation	6
3.	Architectural Goals and Constraints	7
4.	Use-Case View	9
4.1	Use-Case Realizations	9
4.1.1	User Registration	10
4.1.2	User Authentication	10
4.1.3	Recover Password	11
4.1.4	Generate Personalized Trip	12
4.1.5	Modify Trip Plan	12
4.1.6	Budget Tracker	13
4.1.7	Manage tourism Information	14
4.1.8	Verify Tourism Infromation	14
4.1.9	Manage User Profile	15
4.1.10	Send Verification Email	15
4.1.11	Manage Notification Preferences	16
4.1.12	Explore Destinations	17
4.1.13	Manage Subscription	17
4.1.14	View Saved Itineraries	18
4.1.15	Manage Users	19
4.1.16	View System Analytics	19
4.1.17	Fetch Route & Weather Context	20
5.	Logical View	21
5.1	Overview	21
5.2	Architecturally Significant Design Packages	22
6.	Process View	23
6.1	User Registration Activity Diagram	23
6.2	User Login Activity Diagram	25
6.3	User Registration and Login Sequence Diagram	26
6.4	Generate Personalized Trip Activity Diagram	27
6.5	Generate Personalized Trip Sequence Diagram	28
6.6	Modify trip Plan Activity Diagram	29
6.7	Modify Trip Plan Sequence Diagram	30
6.8	Budget Management Activity Diagram	31
6.9	Budget Management Sequence Diagram	32
6.10	Manage Tourism Information Activity Diagram	33

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

6.11	Manage Tourism Information Sequence Diagram	34
6.12	AI Recommendation Flow Activity Diagram	35
7.	Deployment View	35
8.	Implementation View	37
8.1	Overview	37
8.2	Layers	37
9.	Data View	39
10.	Size and Performance	39
10.1	Web Application Loading Time	39
10.2	Mobile App Loading Time	39
10.3	AI Itinerary Generation Requests Per Second	40
10.4	Number of Database Requests Per Second	40
10.5	Number of Simultaneous Users	40
10.6	System Up Time	40
10.7	Resource Utilization	40
11.	Quality	40
11.1	Scalability	40
11.2	Maintainability	41
11.3	Extensibility	41
11.4	Reliability	41
11.5	Portability & Compatibility	41
11.6	Configurability	41
11.7	Attractiveness & Usability	41
12.	References	42

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

Software Architecture Document

1. Introduction

1.1 Purpose

This document provides a comprehensive architectural overview of the system, using a number of different architectural views to depict different aspects of the system. It is intended to capture and convey the significant architectural decisions which have been made on the system.

The document is used as a communication tool between both developers and non-developers. The audience of this document is an individual, team, organization, or classes, having an interest in the realization of the system architecture

1.2 Scope

The document describes the entire architecture of the Green Bay system with appropriate diagrams. Hence, this document helps to understand the software's modules and components without going deep into the code. The document covers 4+1 views and data views of the system. Under the 4+1 views, all aspects of use case view, logical view, process view, deployment view and implementation view are covered in a detailed manner.

1.3 Definitions, Acronyms, and Abbreviations

Acronym/ Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
Gemini AI	Application Programming Interface
Google Maps API	Google service used to provide maps
UI	User Interface
User Authentication	Process of verifying a user's identity
Weather API	External service used to retrieve weather forecasts

Table 1: Definitions, Acronyms and Abbreviations

1.4 References

- **draw.io (Diagrams.net)** – Used for creating system architecture, component, sequence, and database deployment diagrams (<https://app.diagrams.net/>).
- **Lucidchart** – Used for creating conceptual models, process flows, and implementation view diagrams (<https://www.lucidchart.com/>).

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

1.5 Overview

Rest of the document contains ten chapters. Among them, the 2nd and 3rd chapters explain architectural representation, architectural goals and constraints of the system. The next five chapters are reserved to cover the 4+1 views. Hence, use case view, logical view, process view, deployment view and implementation view will be described under those separate chapters. The last three chapters will explain the data view, size, performance, and quality of the system.

2. Architectural Representation

In this document, all the architectures are represented using Unified Model Language[1]. Included architectures are a series of views including use case view, logical view, process view, deployment view and implementation view. Following part will describe the content of each view in 4+1 views

- **Logical View** : Logical view provides an explanation of the structure and the organization of the system. This view is used to describe subset of the classes, subsystems and packages, and use-case realizations. In this document class diagram is included to represent the logical view of the system.
- **Process View** : The process view describes the processes involved in the system's execution and user interactions. Besides, this also shows the allocation of objects and classes to the process. Both sequence and activity diagrams are used to represent the process view of the system.
- **Implementation / Development View**: Implementation view contains two subsections namely overview and layer. This view is used to show how the software is decomposed into layers to do the implementation. It also provides an overview of the allocation of design elements to the implementation. Component diagram and package diagram will be described under the implementation view.
- **Physical View**: The physical view describes how physical networks are connected and configured to deploy and run the system. It also shows the protocols that are used by each component to communicate with each other. Deployment diagram will be used to represent the physical view.
- **Scenarios / Use Case View**: Use Case Diagram is used to represent the scenarios. It shows the actors of the system. And it also shows the tasks that can be done by each actor. Due to these, after exploring use case views, stakeholders can get an overall idea of system functions and scenarios that can be done by each actor.

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

3. Architectural Goals and Constraints

Designers pay attention to safety, security, privacy, reusability, loose coupling and performance. Moreover, designers are concerned about development tools. Because developers can't increase the scalability of developing tools they selected to use if there is a requirement that cannot be handled by the developing tools which they are using.

3.1 Security

User authentication and access control can be used to ensure that only authorized users can access the functionalities related to them. Some functionalities are restricted to specific users. For example, only administrators can add, update, or remove tourism information, while registered users can create and manage their own travel plans. Passwords and other sensitive information should also be handled securely to protect user accounts.

3.2 Privacy

Data provided by users should be kept securely. SmartJourney handles information such as user details, travel preferences, saved itineraries and budget information. These data are stored in the database and should only be accessed by authorized users. Attributes like passwords will be kept as hashed v values. These are important facts to consider in designing.

3.3 Reusability

Using reusable components is helpful for saving time and it will increase the understandability of the code for development. In the application card component is used in several places. Using a more generalized card component will be helpful to reduce the number of lines in coding and it is useful for changing card components from a central place. No need to search for card components everywhere and change that. Changing the centralized card component will do the task. That's why it is important to consider reusable components while designing.

3.4 Loose Coupling

Loose coupling is helpful for maintainability. A requirement for changing components which is connected to several other components will be hard for developers. Therefore designing should consider loose coupling.

3.5 Performance

Users expect high performance and low data usage. As an example there may be thousands of posts loaded when a user navigates to a page. It will be really impacted badly on the system and users. Therefore in the designing phase developers have to think about these issues. AI-based itinerary generation should also process the required travel information efficiently because it involves communication with several external services.

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

3.6 Development Tools

There are several tools, libraries, and services that can be used to develop the SmartJourney system. When selecting these technologies, it is important to consider whether they support the required functionalities and future scalability of the system. The main development tools and technologies used for this system are:

- Web Frontend – React
- Mobile Frontend – React Native
- Backend – Spring Boot
- Database – PostgreSQL
- AI Recommendation – Gemini AI
- Location Service – Google Maps API
- Weather Information – Weather API

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

4.1.1 User Registration

Use case name	User registration
Actor	Unregistered user
Description	Unregistered users can sign up to the system to be eligible for all features provided by the system.
preconditions	The user is not enter already registered email to the system
Main flow	1. User visits the signup page 2. Users fill the form details 3. Submit the details
Successful end/post condition	User details are successfully saved to the system. And send a credential email to the user
Fail end/post condition	System will prompt an error message that field data has something wrong or cannot be null
Extensions	System sends a credential email to the user

Table 2: User Registratio

4.1.2 User Authentication

Use case name	User Authentication
Actor	Registered user, Authority member
Description	After signing up to the system users can sign in to the system with password and username
preconditions	The user should have an user account
Main flow	4. User visits the sign in page 5. Users fill the sign in details 6. Submit the details
Successful end/post condition	Redirect to the landing page, successful Authentication by the system and return a token to the user's browser
Fail end/post condition	System prompt an error message that password or username has an error or incorrect

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

Extensions	User authenticates using Google sign-in User enables Remember Me.
-------------------	--

Table 3: User Authentication

4.1.3 Recover Password

Use case name	Recover Password
Actor	Registered User
Description	Allow a registered user to securely reset a forgotten password using a verification email
preconditions	Users need to have registered account
Main flow	1. User selects “Forgot Password” 2. The system requests the email address 3. The system verifies the account 4. The system generates a password reset token 5. The system sends a password reset email 6. User opens the reset link 7. User enters a new password 8. The system validates and updates the password
Successful end/post condition	The account password is updated successfully
Fail end/post condition	Token expired Email not registered
Extensions	User be able to change password anytime

Table 4: Recover Password

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

4.1.4 Generate Personalized Trip

Use case name	Generate Personalized Trip
Actor	Registered user
Description	Users give travel preferences such as destination, travel dates, budget, travel style, and personal interests to generate weather forecasts, travel routes and customized itinerary that can be reviewed, modified and saved.
preconditions	The user needs to register to the platform to create, review, modify and save itinerary.
Main flow	1. User selects Create New Trip 2. User enters travel preferences 3. User reviews the itinerary 4. Users modify the itinerary if there any changes exist 5. User saves the itinerary
Successful end/post condition	A personalized itinerary is generated successfully and stored in the user's account
Fail end/post condition	Show error message if required information are not provided
Extensions	The user can modify preferences and regenerate the itinerary before saving it.

Table 5: Personalized Trip Generation

4.1.5 Modify Trip Plan

Use case name	Modify Trip Plan
Actor	Registered User
Description	User can modify an existing itinerary after it has been generated by request destination changes, remove activities, change the travel route, add new places or changing budget using conversation with AI assistant
preconditions	User should have an existing itinerary

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

Main flow	1. User opens a saved itinerary 2. User requests modification through the AI chat 3. User checks the updated itinerary 4. User saves the modified itinerary
Successful end/post condition	The itinerary is modified successfully and replaces the previous version
Fail end/post condition	N/A
Extensions	Users could be able to repeatedly modify the itinerary until the daily tokens finish.

Table 6: Modify Trip Plan

4.1.6 Budget Tracker

Use case name	Budget Tracker
Actor	Registered User
Description	Users monitor and manage their travel expenses on active trips.
preconditions	Users need to log in to the system and should have at least one saved itinerary with a defined budget
Main flow	9. User navigates to the “Budget Tracker” page and select the specific itinerary 10. User clicks the “+Add expense” button and inputs a transaction date, description, category, and amount
Successful end/post condition	User budget tracker updated successfully
Fail end/post condition	Showing error message for invalid amount inputs
Extensions	User allows to add expenses and get updated with the budget status Export expense report Receive budget alert

Table 7: Budget Tracker

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

4.1.7 *Manage tourism Information*

Use case name	Manage Tourism Information
Actor	Administrator
Description	Administrators manage hotels, attractions, and events while ensuring that only verified information is available for itinerary generations.
preconditions	The administrator should log into the administrator panel
Main flow	1. Administrator selects a tourism category 2. Administrator chooses to add, edit or remove information 3. Administrator enters or updates the required information
Successful end/post condition	Tourism information is updated successfully
Fail end/post condition	Database error
Extensions	N/A

Table 8: Manage Tourism Informations

4.1.8 *Verify Tourism Infromation*

Use case name	Verify Tourism Information
Actor	Administrator
Description	Administrators enable to review and approve submitted tourism information
preconditions	Pending tourism information exists
Main flow	4. Administrator opens the verification queue 5. Administrators review the information 6. Administrators approve or reject the record

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

Successful end/post condition	Tourism information is verified successfully
Fail end/post condition	Database error
Extensions	N/A

Table 9: Verify User Info

4.1.9 Manage User Profile

Use case name	Manage User Profile
Actor	Registered User
Description	Users manage their personal account information according to their preferences by updating profile details, changing passwords, changing notification settings, and managing travel preferences.
preconditions	Users need to log in to the system
Main flow	11. Users open the profile page 12. Users update the required information 13. Users click the save changes button
Successful end/post condition	User profile information is updated successfully
Fail end/post condition	N/A
Extensions	User allows to change the passwords, notification settings and location setting anytime.

Table 10: Manage User Profile

4.1.10 Send Verification Email

Use case name	Send Verification Email
Actor	System
Description	When an unregistered user signs up to the system, an auto generated password is sent to the email provided by the user by the system.

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

preconditions	Users need to give active and correct email
Main flow	1. The system generates a unique verification token 2. The system stores the verification token 3. The system prepares the verification email 4. The system sends the email through the email service
Successful end/post condition	System successfully sent verification code through email
Fail end/post condition	Prompt an error message showing invalid email address
Extensions	N/A

Table 11: Send Verification Email

4.1.11 Manage Notification Preferences

Use case name	Manage Notification Preferences
Actor	Registered User
Description	Allow registered users to configure notification settings for trip reminder, Weather alerts and email notifications
preconditions	N/A
Main flow	1. User opens notification settings 2. User modifies the required settings 3. User confirms the update
Successful end/post condition	Notification settings are updated successfully
Fail end/post condition	User should register to the application
Extensions	N/A

Table 12: Manage Notification

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

4.1.12 Explore Destinations

Use case name	Explore Destinations
Actor	Unregistered User
Description	Allows unregistered users to browse and search popular destinations, attractions, and sample travel routes without logging in.
preconditions	N/A
Main flow	1. User opens the destination explore page. 2. User searches or filters destinations by category, region, or popularity. 3. System renders destination details and recommendations.
Successful end/post condition	Selected destination details are displayed to the user.
Fail end/post condition	System shows an error message for failed content loading or empty search results.
Extensions	User clicks a prompt to sign up or register to save a destination.

Table 13:Explore

4.1.13 Manage Subscription

Use case name	Manage Subscription
Actor	Registered User
Description	Enables registered users to view, upgrade, downgrade, or cancel subscription plans for premium AI trip features.
preconditions	User must be logged into an active account.
Main flow	1. User navigates to subscription settings. 2. User selects a target plan and confirms billing details. 3. System processes payment and updates

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

	user tier permissions.
Successful end/post condition	Account tier is updated successfully with expanded token/trip limits.
Fail end/post condition	Payment gateway failure or transaction decline error displayed.
Extensions	User cancels auto-renewal or views payment transaction history.

Table 14: Manage Subscription

4.1.14 View Saved Itineraries

Use case name	View Saved Itineraries
Actor	Registered User
Description	Allows users to access, view, filter, and organize previously generated and saved trip plans.
preconditions	User must be logged in and have at least one saved itinerary.
Main flow	1. User opens the "My Trips" / Saved Itineraries section. 2. System displays a list of all saved trip plans. 3. User selects a specific trip to view full itinerary details.
Successful end/post condition	Selected itinerary details are displayed successfully.
Fail end/post condition	Unable to retrieve itineraries due to network/server failure.
Extensions	User filters or sorts itineraries by date, status, or destination.

Table 15: View Saved Itineraries

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

4.1.15 Manage Users

Use case name	Manage Users
Actor	Admin
Description	Allows administrators to oversee user accounts, modify user roles, and deactivate or resolve suspended accounts.
preconditions	Administrator must be authenticated into the admin portal.
Main flow	1. Administrator accesses the user management module. 2. Administrator searches for a user by email, name, or ID. 3. Administrator updates status, role, or user parameters and confirms changes.
Successful end/post condition	Target user account state is successfully updated in the database.
Fail end/post condition	System prompts a database error or invalid account action notification.
Extensions	Administrator exports user activity logs.

Table 16: Manage Users

4.1.16 View System Analytics

Use case name	View System Analytics
Actor	Admin
Description	Enables administrators to monitor system performance, user engagement, AI token usage metrics, and booking trends.
preconditions	Administrator must be logged into the admin portal.
Main flow	1. Administrator selects the analytics dashboard. 2. Administrator specifies time ranges or metric filters. 3. System generates and displays usage

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

	graphics and statistical reports.
Successful end/post condition	Real-time analytics report is loaded and displayed.
Fail end/post condition	Failed to retrieve data metrics from the analytics server.
Extensions	Administrator exports system analytics summaries to CSV or PDF.

Table 17: View System Analytics

4.1.17 Fetch Route & Weather Context

Use case name	Fetch Route & Weather Context
Actor	External Services
Description	Automatically fetches real-time weather forecasts and routing options from third-party APIs during trip generation.
preconditions	Automatically triggered by the "Generate Personalized Trip" process.
Main flow	1. System issues API requests to external routing and weather service providers. 2. External services evaluate travel coordinates and dates. 3. External services return weather context and route information to the system.
Successful end/post condition	Weather context and route paths are integrated into the trip context.
Fail end/post condition	Third-party API timeout occurs; system falls back to cached or generalized data.
Extensions	N/A

Table 18: Weather Routings

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

5. Logical View

5.1 Overview

The whole system is divided into five sections namely user management, content management, itinerary management, budget management, and AI service management. Among them, the AI service management will not be stored in the database. However, the AI service management is responsible to get the required natural language queries from the user, use the Gemini API and LangChain framework to process the data, and generate personalized travel itineraries.

Content management is responsible for managing travel listings that are verified and published by administrator users. All the verified travel listings are stored in the database.

User management is responsible for handling users. The system supports three types of users namely unregistered users, registered travelers, and administrator users. Among them both registered travelers' details and administrator users' details are stored in the database. Moreover, these users have different capabilities as mentioned in the functional requirements.

Itinerary management modules handle saved travel plans. Saved itineraries are only visible to the owner of the itinerary. However, the verified destination data used to build these itineraries is visible to all the system users. Verified travel listings are only publishable by administrator users. Both itineraries and travel listings are stored in the database.

Budget management handles both total trip budgets and individual expenses recorded in the system. Total budgets are defined in the system by registered travelers and individual expenses are published by the travelers to calculate the remaining budget.

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

5.2 Architecturally Significant Design Packages

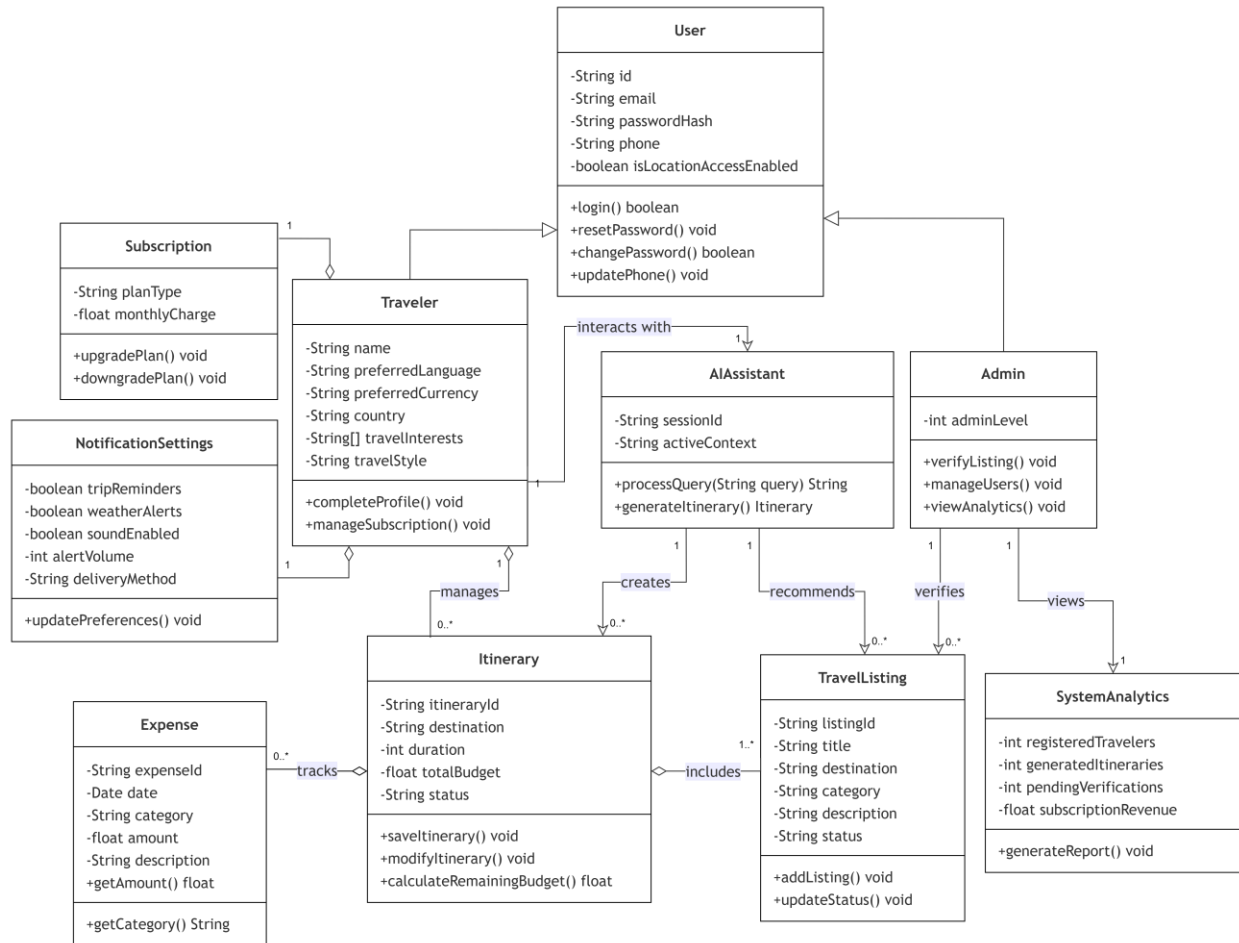


Figure 2: Class Diagram

The class diagram of the system contains 10 classes. User class uses a password hash and authentication logic to authenticate the registered users. User class has two child classes which represent travelers and administrator users. Administrator users should have an admin level. Moreover, these users can verify travel listings to be viewed by the public.

Travelers can communicate natural language queries to the AIAssistant. And personalized travel plans for these queries are generated by the AIAssistant. System has an itinerary management and budget tracking facility. This is handled by the Itinerary and Expense classes. Both Itinerary and Expense are linked via composition, meaning expenses cannot exist without a parent itinerary. However, itineraries are generated and saved by travelers, and verified travel listings are published by administrator users.

All the verified travel listings, including hotels, restaurants, and attractions, can be viewed by the public. Furthermore, travelers possess personal configurations managed by the

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

Subscription and NotificationSettings classes, which are strictly composed within the Traveler class.

System uses the AIAssistant class to interpret user prompts and generate itineraries. This assistant uses session IDs and active context while orchestrating generation. The system has various types of administrative data namely registered travelers, generated itineraries, and subscription revenue. Compilation of these metrics is handled by the SystemAnalytics class, which is accessed directly by the Admin class.

The system can notify travelers if a specific event such as an upcoming trip reminder, budget limit, or weather alert happens. To handle this scenario, the system uses the NotificationSettings class to store the preferred delivery methods and alert toggles. Travelers are the recipients, and the system delivers updates based on these configuration states.

It is necessary to optimize data redundancy within the system. Due to this, the system generates aggregated Itinerary objects that simply reference existing TravelListing objects in the database rather than duplicating the destination data. This structural relationship is handled by an aggregation between the Itinerary and TravelListing classes.

6. Process View

6.1 User Registration Activity Diagram

The user registration process allows a new traveller to create a SmartJourney account before accessing personalized travel planning features. Initially, an unregistered user visits the registration page and enters the required personal information including name, email address and password.

After the registration form is submitted, the Authentication Controller validates all input fields. If any validation errors are detected, the system requests the user to correct the invalid information before continuing.

Once the submitted information is successfully validated, the Authentication Service creates a new user account and securely stores the user information in the database with an encrypted password. After that, the Email Service generates an email verification token and sends a verification email to the registered email address. After successful email delivery, the system notifies the user that registration has been completed successfully and redirects the user to the login page.

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

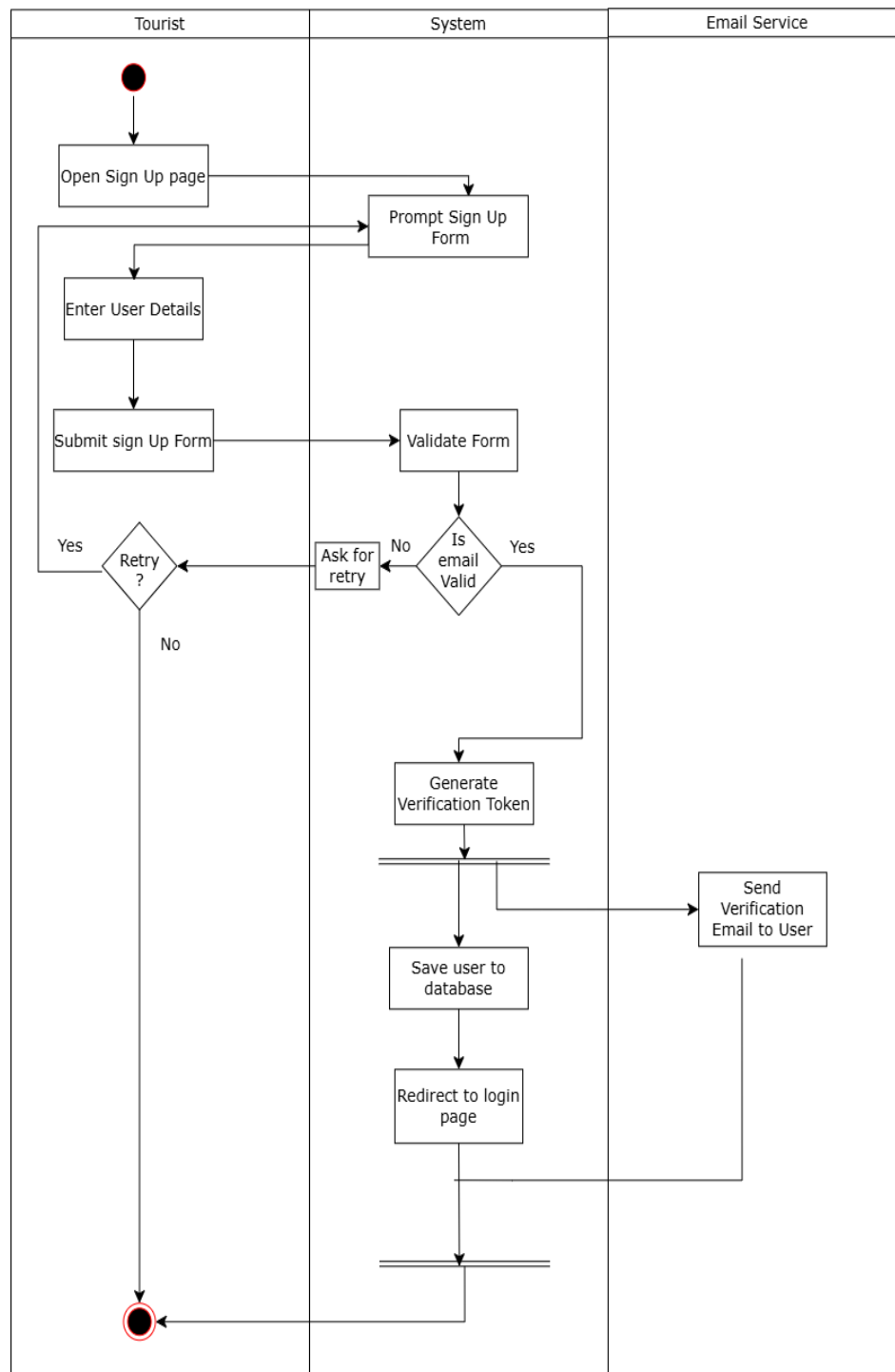


Figure 3: User Registration Activity Diagram

6.2 User Login Activity Diagram

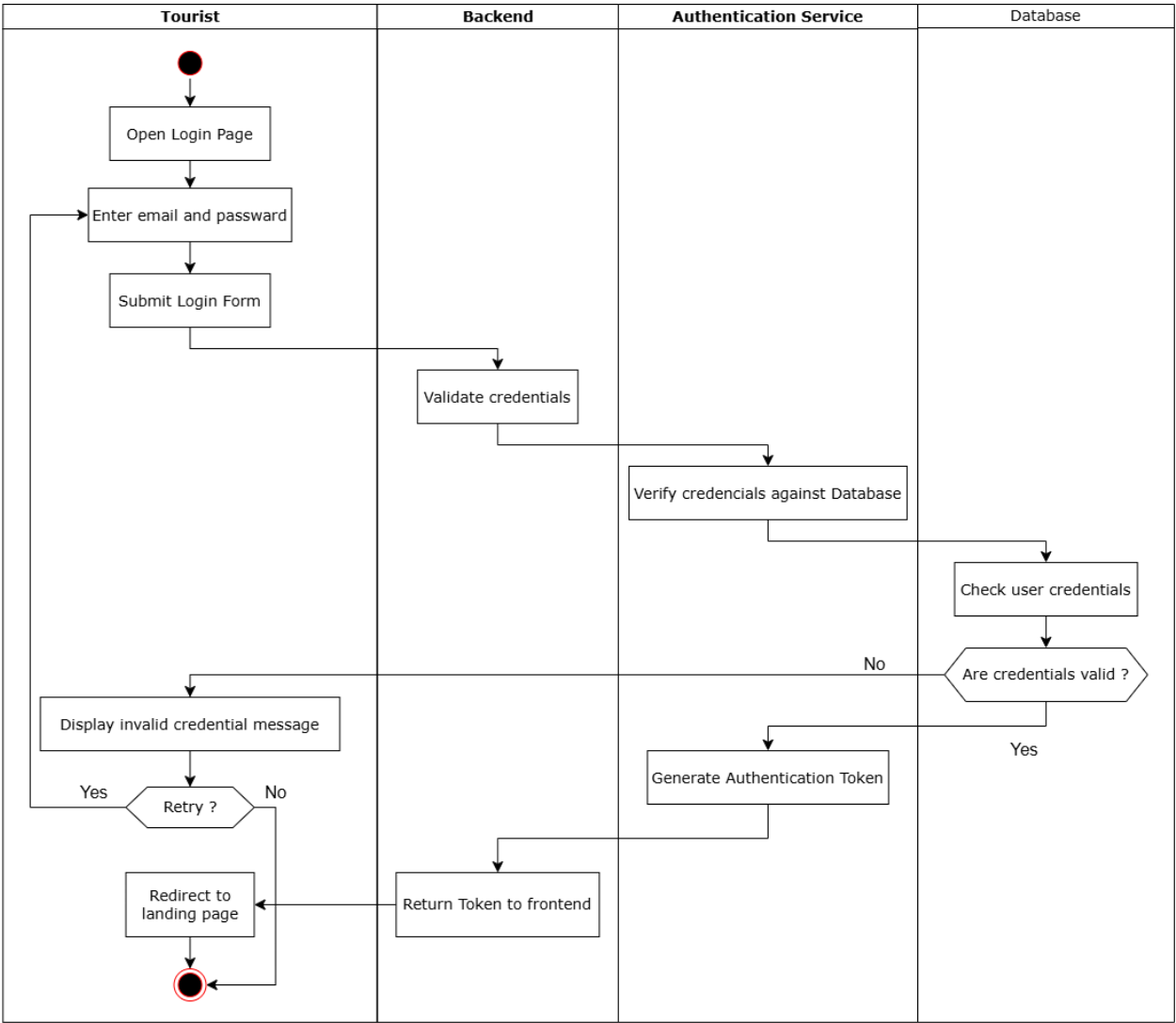


Figure 4: User Login Activity Diagram

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

6.3 User Registration and Login Sequence Diagram

Initially, the Registration Interface sends the user's registration details to the Authentication Controller. The controller validates the submitted information before forwarding the request to the User Service. The User Service stores the new user account in the database and requests the Email Service to generate and send the verification email.

Once the email has been successfully sent, a confirmation message is returned through the Authentication Controller to the Registration Interface, where the user is informed that the registration process has completed successfully.

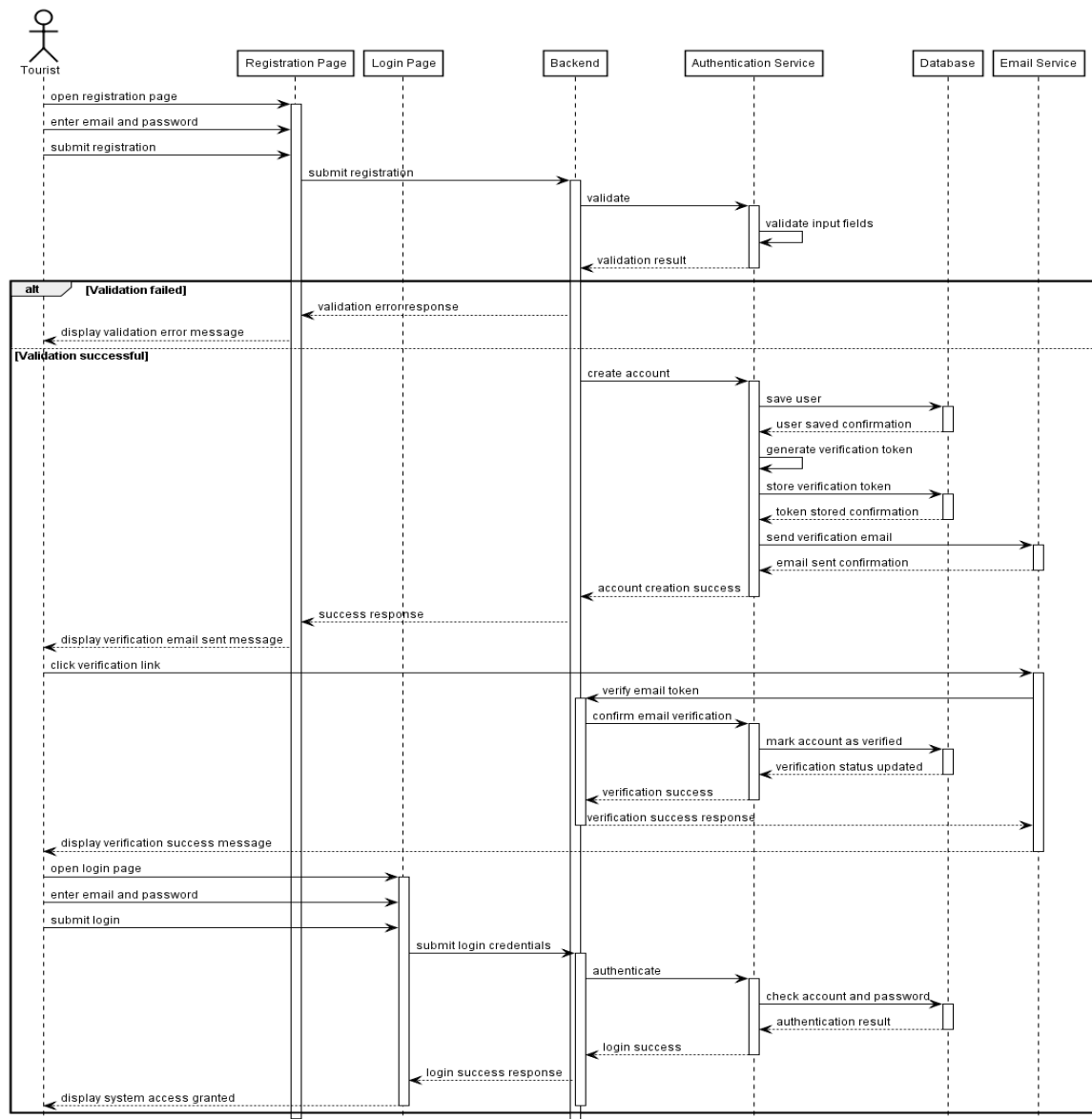


Figure 5: User Registration and Login Sequence Diagram

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

6.4 Generate Personalized Trip Activity Diagram

After logging into the system, the user selects the option to create a new trip and provides travel preferences such as destination, travel dates, budget, accommodation preferences, transportation methods and personal interests.

The system validates the information provided before forwarding the request to the AI Trip Planning Service. The AI service communicates with external APIs including Google Maps API, Weather API and Gemini AI to obtain route information, weather forecasts and intelligent travel recommendations.

After gathering all required information, the AI engine generates a personalized itinerary containing daily schedules, attractions, accommodations, transportation suggestions and estimated expenses. The generated itinerary is displayed to the user, who may either save it directly or continue modifying the plan.

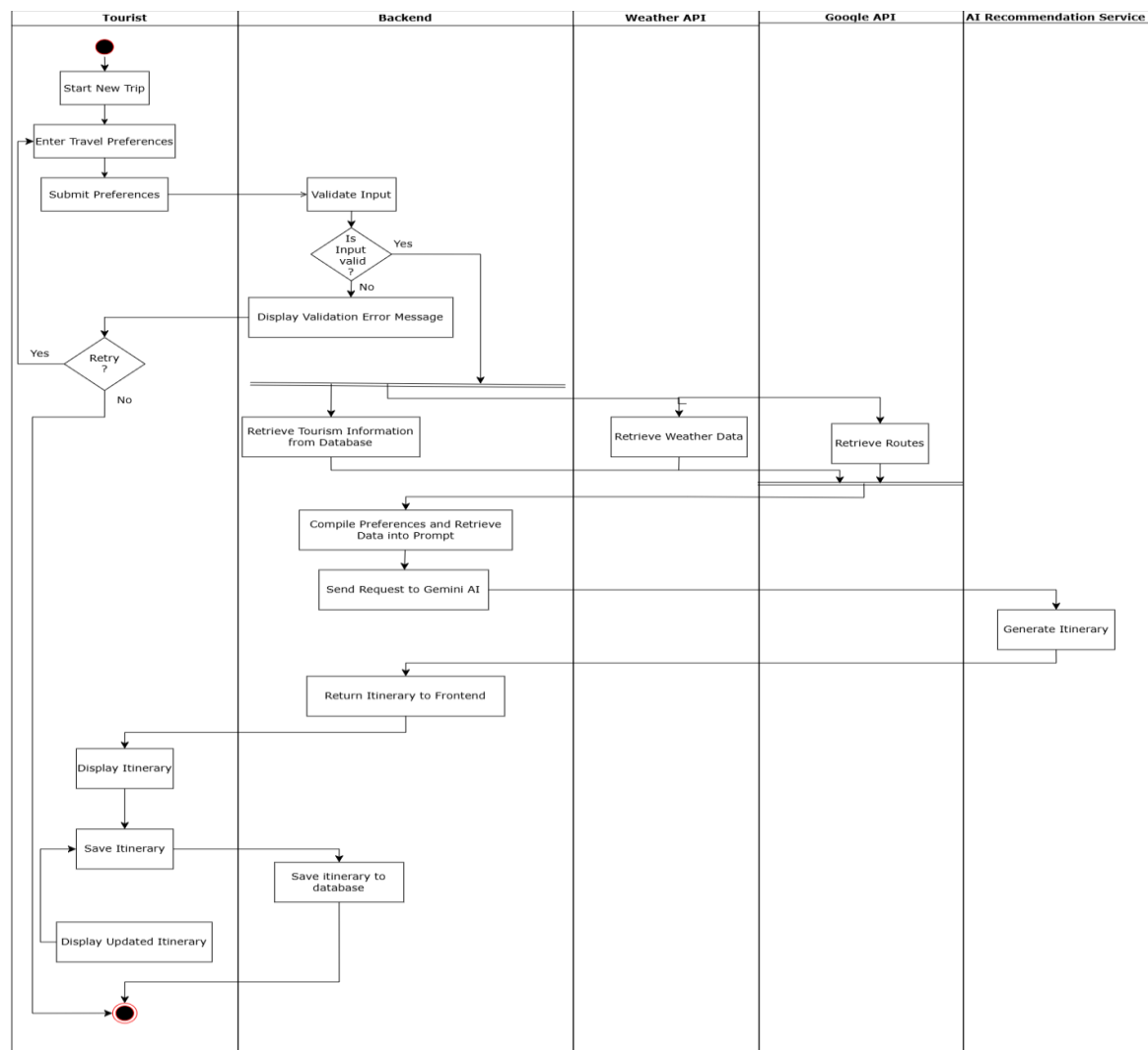


Figure 6: Generate Personalized Trip Activity Diagram

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

6.5 Generate Personalized Trip Sequence Diagram

The process begins when the tourist submits travel preferences through the trip planning interface. The frontend forwards the request to the backend, where the information is validated before being processed. The backend communicates with the AI Recommendation Service, which retrieves verified tourism information from the database together with route details from the Google Maps API and weather forecasts from the Weather API. Using the collected information and user preferences, the AI service generates a personalized itinerary and returns it to the backend. The generated itinerary is then stored in the database and sent to the frontend for presentation. The tourist can review the itinerary, save it for future use or request further modifications through the AI assistant.

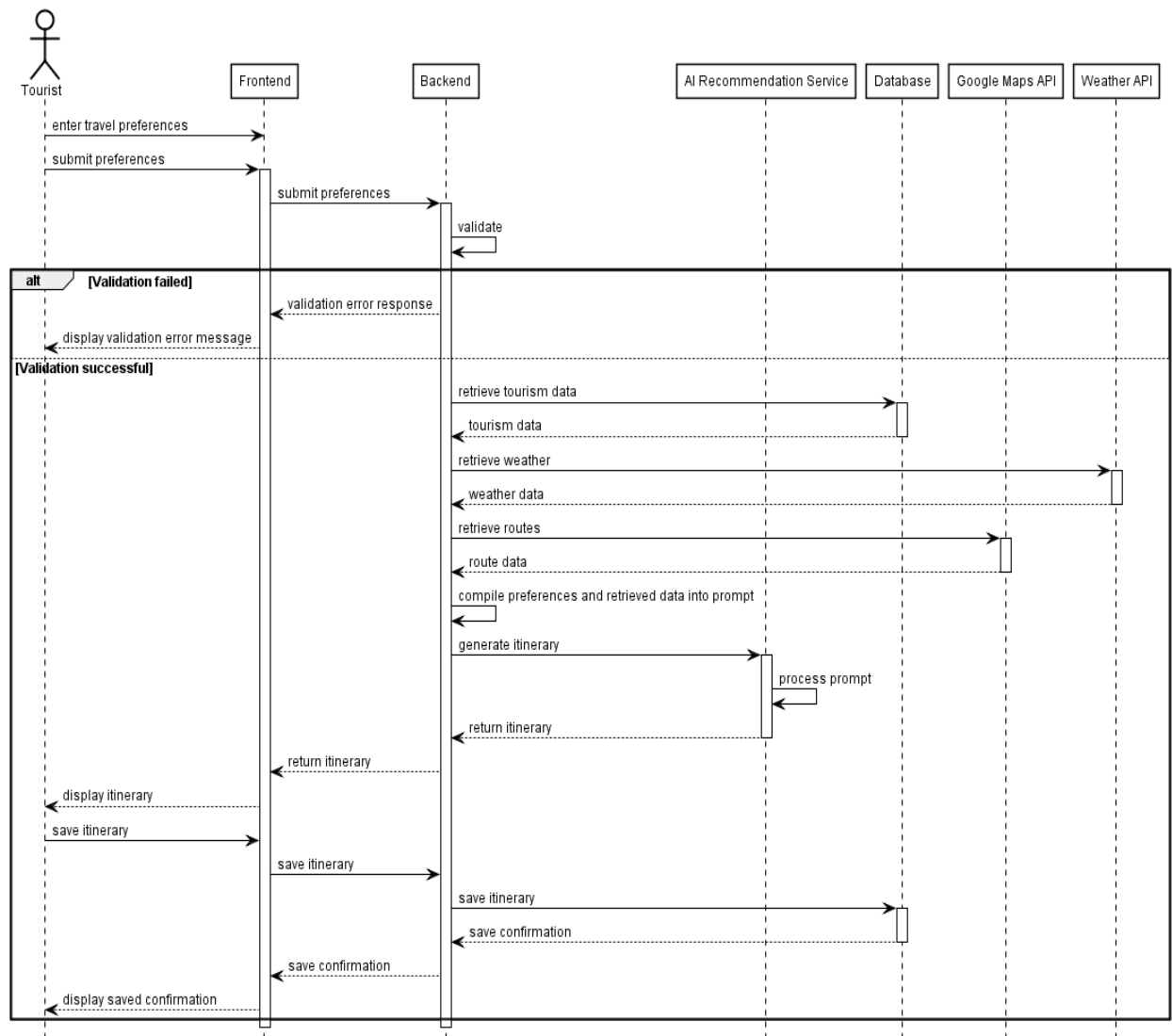


Figure 7: Generate Personalized Trip Sequence Diagram

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

6.6 Modify trip Plan Activity Diagram

The process begins when the tourist selects a previously saved itinerary and submits modification requests through the AI travel assistant. The system retrieves the selected itinerary and validates the requested changes before forwarding them to the AI Recommendation Service. Based on the updated preferences, the AI regenerates the itinerary while preserving relevant travel information. The modified itinerary is then displayed to the tourist for review. The tourist may either continue making further modifications or save the updated itinerary, after which the revised itinerary is stored in the database for future access.

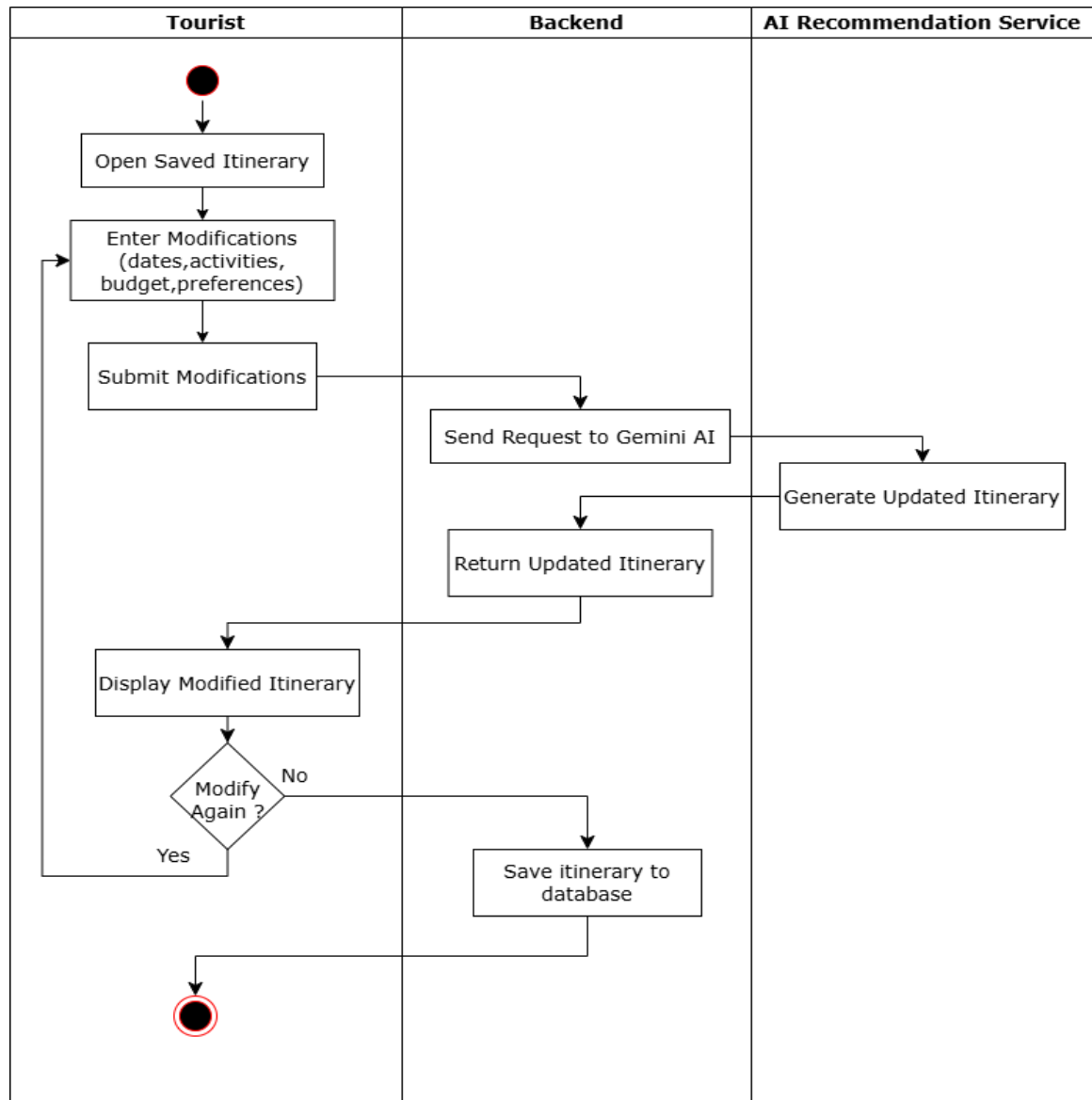


Figure 8: Modify trip Plan Activity Diagram

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

6.7 Modify Trip Plan Sequence Diagram

The process begins when the tourist selects a saved itinerary and submits the required modifications through the trip planning interface. The frontend forwards the request to the backend, where the existing itinerary is retrieved from the database, and the submitted changes are validated. The backend then communicates with the AI Recommendation Service to regenerate the itinerary based on the updated travel preferences. After the itinerary has been successfully regenerated, the updated version is stored in the database and returned to the frontend. Finally, the modified itinerary is displayed to the tourist, allowing further modifications or saving the final travel plan

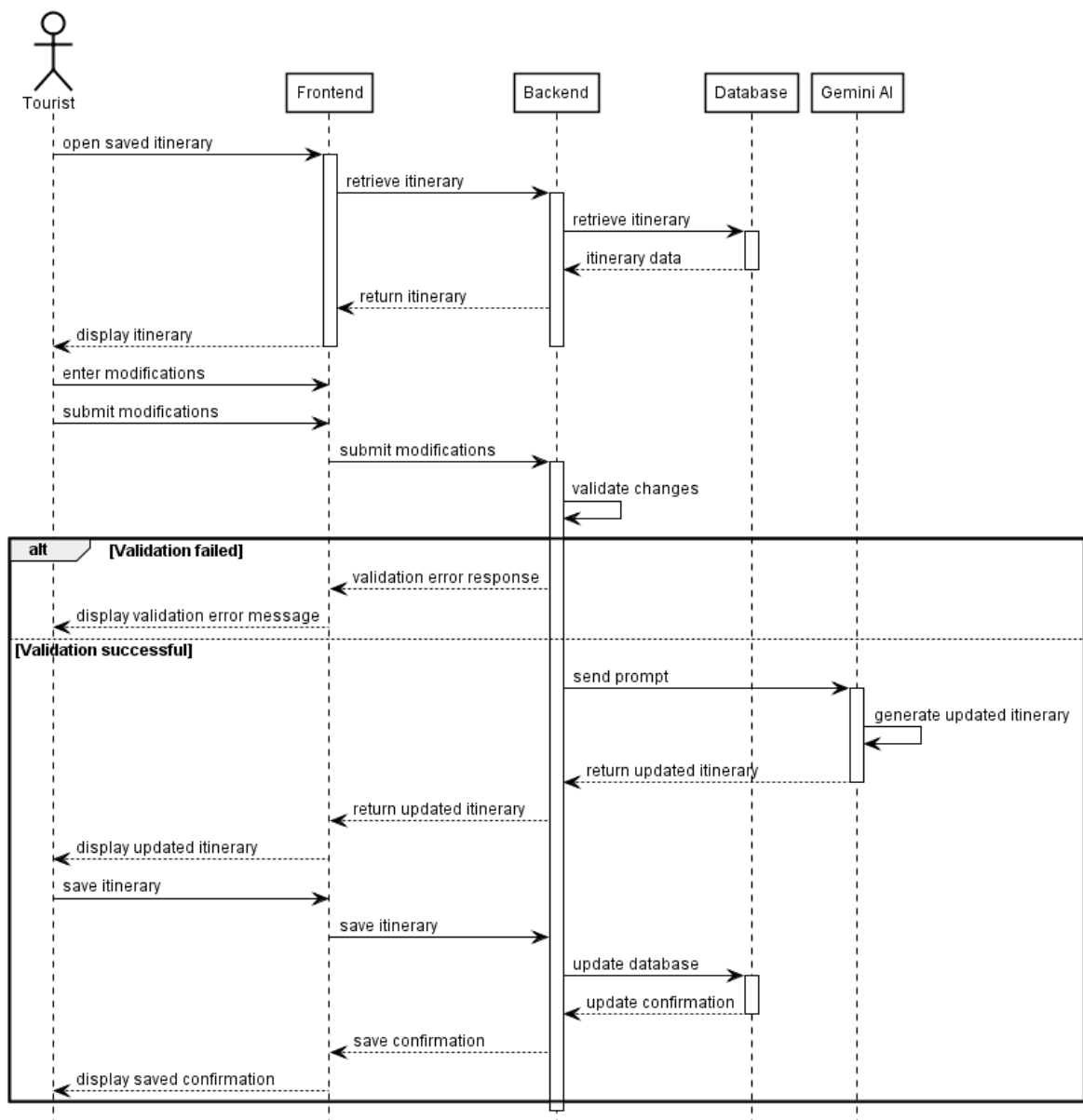


Figure 9: Modify Trip Plan Sequence Diagram

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

6.8 Budget Management Activity Diagram

When the tourist accesses the Budget Tracker from an existing itinerary. The user can enter or update estimated expenses for different categories such as accommodation, transportation, meals and activities. The system validates the submitted expense information before updating the budget details in the database. Once the calculations are completed, the remaining budget and expenditure summary are displayed to the tourist. The user may continue updating expenses or return to the itinerary planning page to make further travel arrangements.

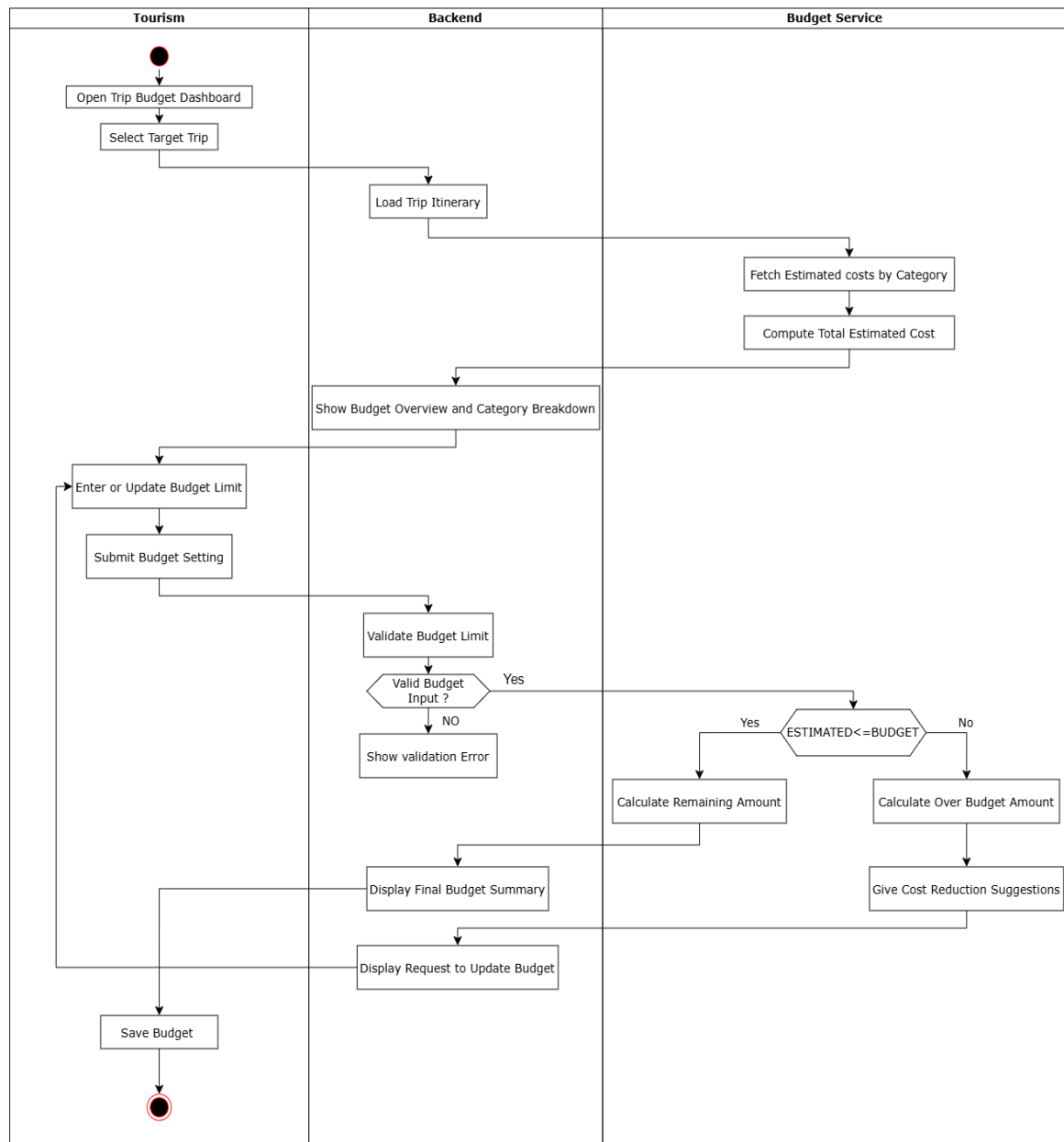


Figure 10: Budget Management Activity Diagram

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

6.9 Budget Management Sequence Diagram

Tourist accesses the Budget Tracker and submits expense details through the frontend application. The frontend forwards the request to the backend, where the submitted information is validated before being processed. The backend retrieves the current budget information from the database, updates the recorded expenses, and recalculates the remaining budget. After the updated budget details are successfully stored in the database, the backend returns the revised budget summary to the frontend. Finally, the updated expenditure details and remaining budget are displayed to the tourist, allowing further expense management throughout the trip.

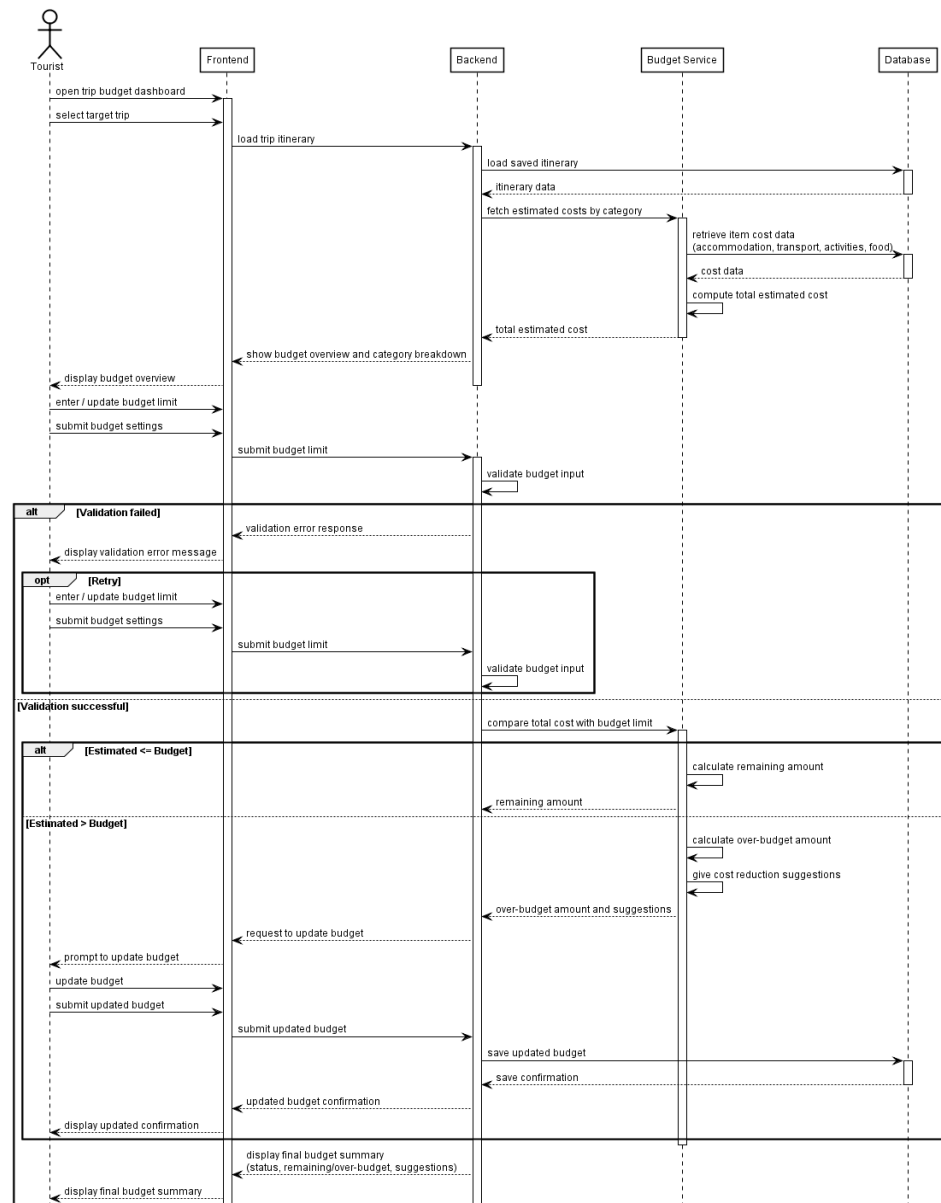


Figure 11: Budget Management Sequence Diagram

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

6.10 Manage Tourism Information Activity Diagram

The administrator logs into the administration portal and selects the tourism information management module. The administrator can add new destinations, hotels, restaurants, attractions or events, as well as update or remove existing records. Before applying the requested changes, the system validates the submitted information to ensure its accuracy and completeness. After successful validation, the updated information is stored in the database and becomes available for the AI Recommendation Service to generate personalized travel itineraries. If validation fails, the administrator is prompted to correct the information before resubmitting the request.

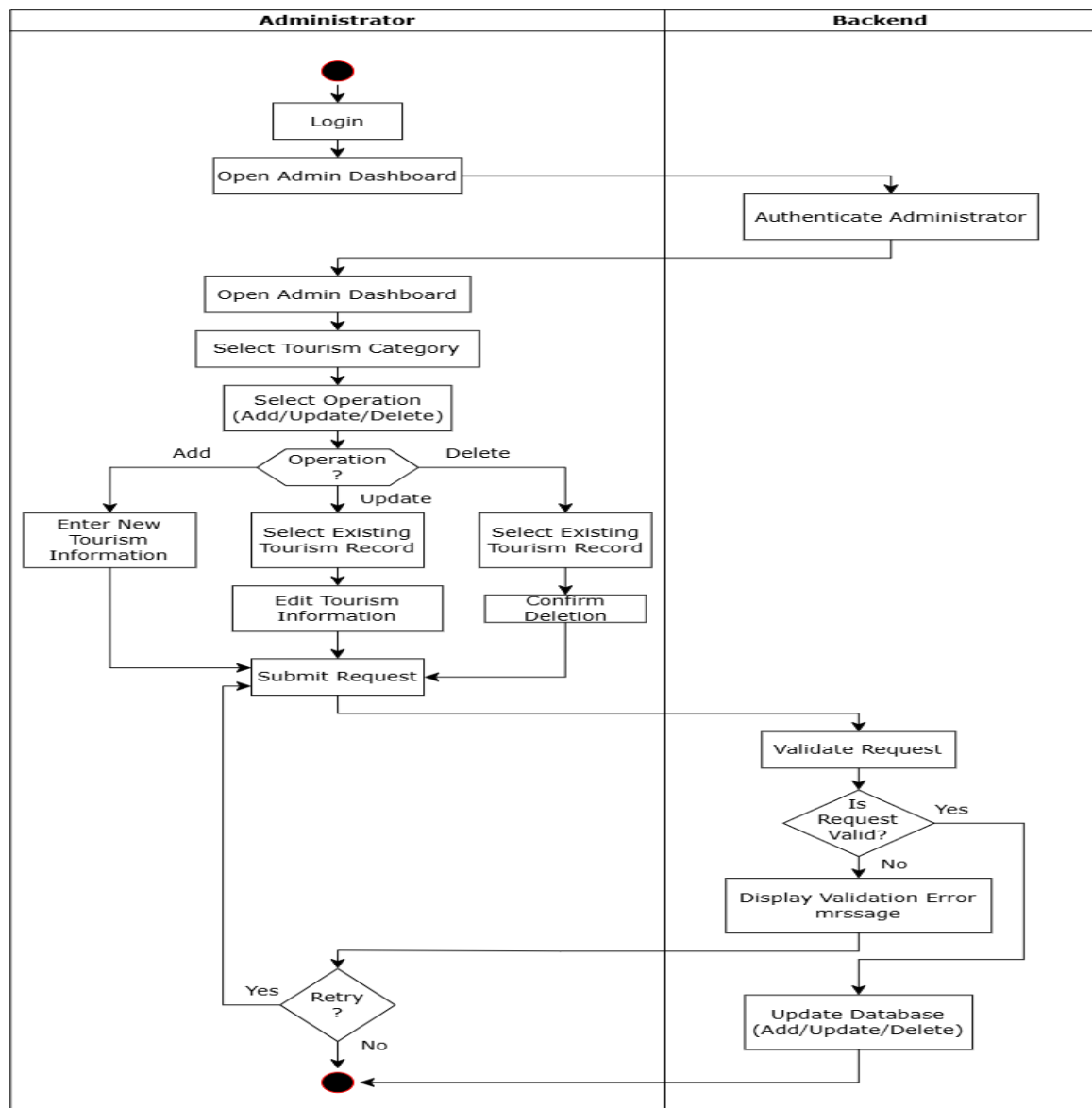


Figure 12: Manage Tourism Information Activity Diagram

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

6.11 Manage Tourism Information Sequence Diagram

The administrator submits a request to add, update or remove tourism information through the administration interface. The frontend forwards the request to the backend, where the administrator's authorization and the submitted information are validated. Once the validation process is completed successfully, the backend updates the tourism records stored in the database. After the operation is completed, the backend returns a confirmation response to the frontend, and the updated tourism information becomes available for the AI Recommendation Service to generate personalized travel recommendations.

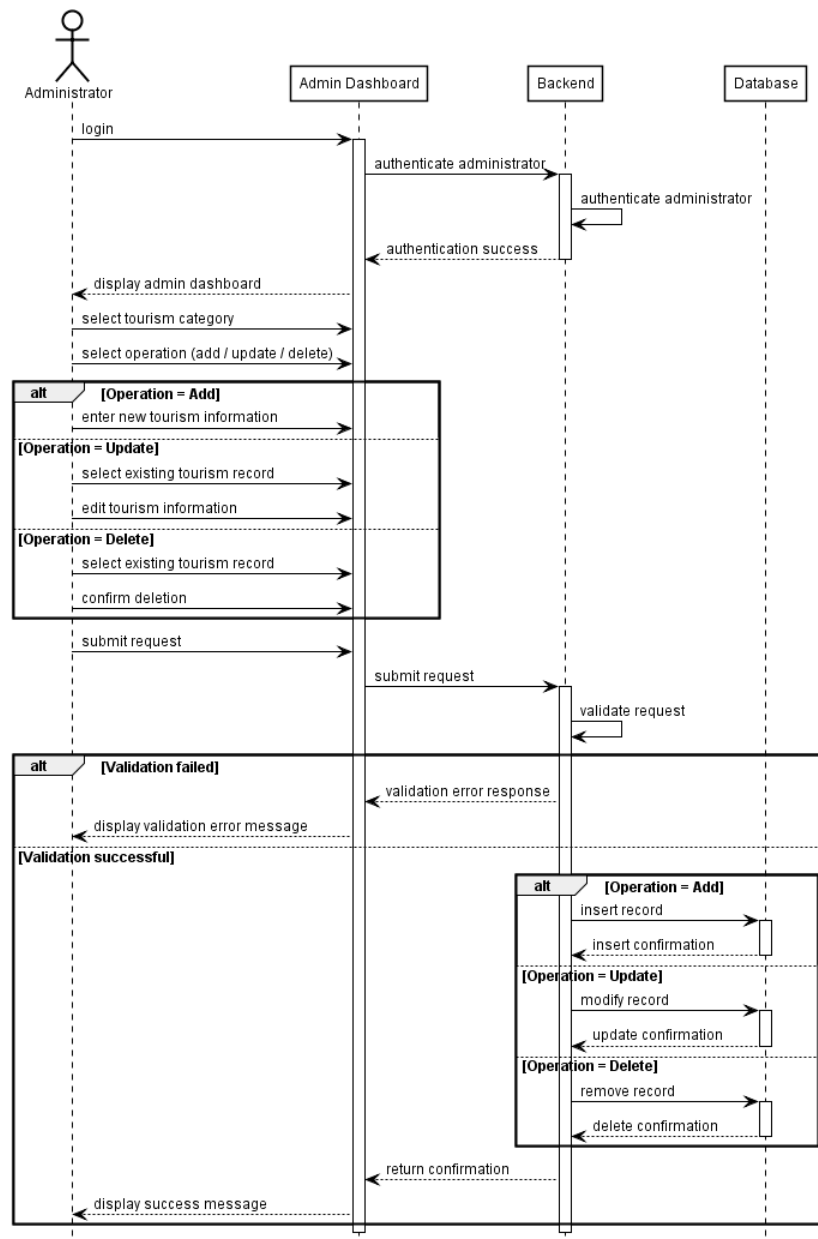


Figure 13: Manage Tourism Information Sequence Diagram

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

6.12 AI Recommendation Flow Activity Diagram

The process begins when the tourist submits travel preferences through the trip planning interface. The system validates the submitted information and retrieves the required travel data from the tourism database and external services. The AI Recommendation Service then generates a personalized travel itinerary based on the collected information.

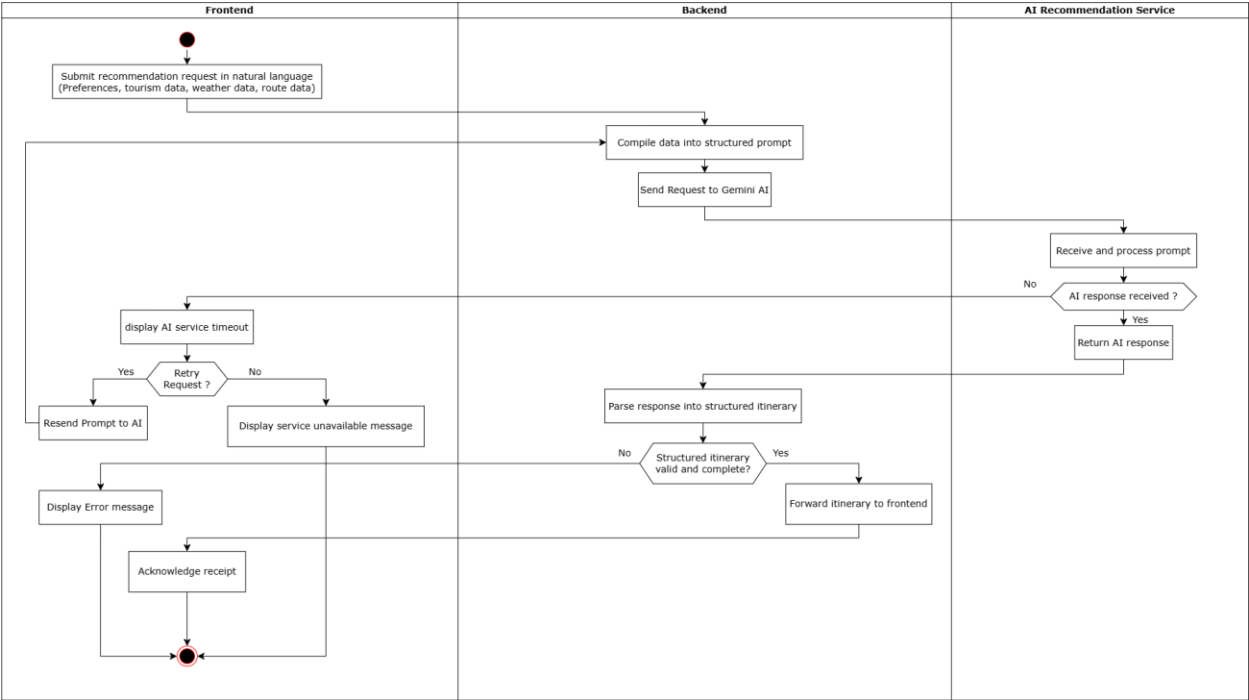


Figure 14: AI Recommendation Flow Activity Diagram

7. Deployment View

This section describes the physical hardware configurations and network interconnections for the SmartJourney platform's deployment model. The system utilizes a cloud-based client-server architecture distributed across user devices and internal servers hosted on Amazon Web Services (AWS). On the client side, users access the system via a PC running a Web Browser execution environment for the Next.js Web Application, or a Mobile Device running the native iOS/Android OS for the Flutter App. These client nodes connect securely via HTTPS to the AWS Cloud infrastructure. Within the cloud, the architecture is divided into specialized execution environments:

- Frontend Server hosting the web components
- Application Server running NestJS for core application logic and REST APIs
- Dedicated AI Server running Python and FastAPI for the Agentic AI System
- Database Server running PostgreSQL via Amazon RDS.

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

The cloud servers communicate with each other and external client devices over HTTPS. Application and AI servers connect directly to the PostgreSQL database using standard TCP/IP protocols. The system integrates via HTTPS with external services such as the Gemini API, Google Maps, OpenWeather, Amazon SES, and Firebase Cloud Messaging, to support core platform functionality and user notifications.

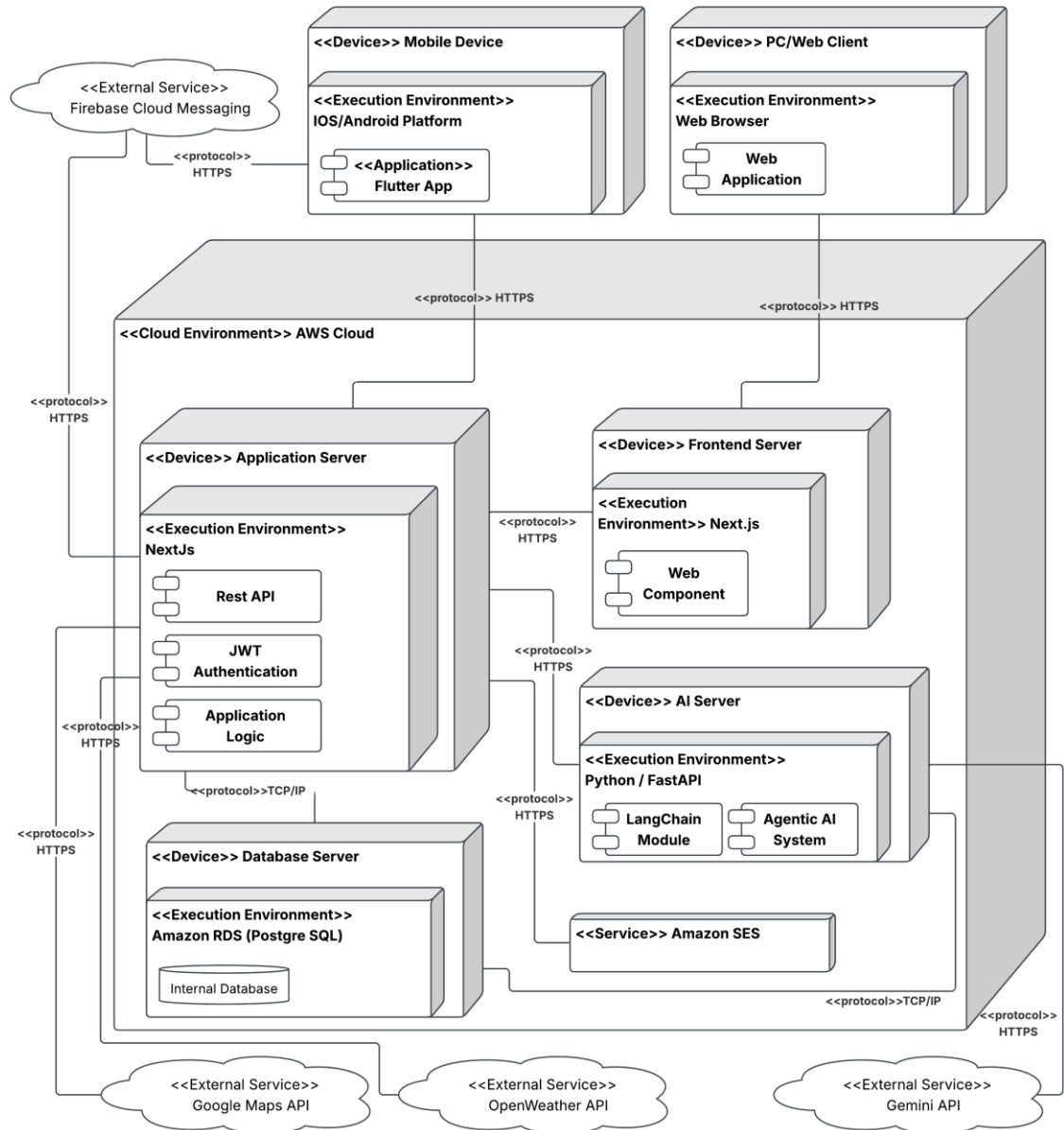


Figure 15: Deployment Diagram

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

8. Implementation View

8.1 Overview

The implementation model of SmartJourney is organized into three separate architectural layers to make development and maintenance easier. The system uses a client-server structure along with a separate AI service layer. The three main layers are the Presentation Layer, the Application Layer, and the Data Layer. The Presentation Layer only handles the user interface and communicates with the Application Layer using secure web requests. The Application Layer processes all the business rules, manages the AI logic, and securely talks to the Data Layer. Finally, the Data Layer handles all database storage and can only be accessed through the Application Layer.

8.2 Layers

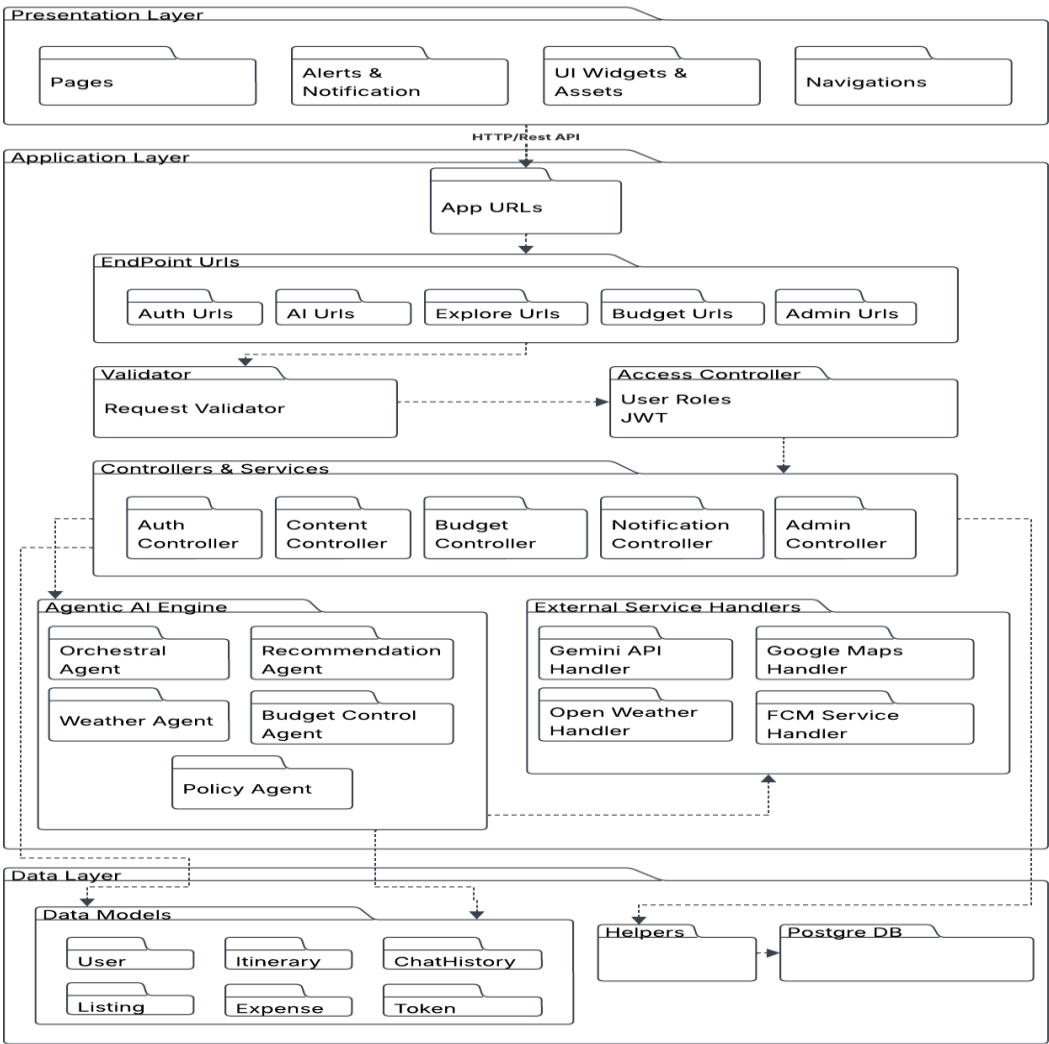


Figure 16: Package Diagram

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

Separating the system into distinct layers helps improve the efficiency of development and maintenance tasks. Because of this, the entire system is divided into three layers that are developed separately and combined to create the final product.

Presentation Layer The Presentation Layer is responsible for the user interface provided to the client and directly impacts the user experience. It includes the web application built with Next.js and the mobile application built with Flutter. This layer contains all the visual pages and navigation menus for users and administrators, such as login screens, chat interfaces, destination exploration feeds, and budget trackers. It also manages user input forms, interactive tools like route maps, and visual alerts for weather or budget warnings.

Application Layer The Application Layer handles the core business logic of the entire application by processing requests from the Presentation Layer and sending back the appropriate responses. It checks user permissions, validates incoming data, and manages secure logins using JSON Web Tokens. This layer includes the main NestJS backend for managing user accounts and travel content, as well as an independent Agentic AI System built with Python, FastAPI, and LangChain. The AI system uses specialized agents to understand user requests and automatically create personalized travel itineraries. Additionally, this layer is responsible for communicating with external services like the Gemini API, Google Maps, OpenWeather, and Firebase Cloud Messaging.

Data Layer The Data Layer is designed to manage all the database operations of the system. Direct access to this layer is restricted, meaning all requests must safely come through the Application Layer. This layer handles the creation, reading, updating, and deleting of all system records, including user profiles, saved travel itineraries, and verified tourism locations like hotels and restaurants. It is built using a PostgreSQL database hosted on Amazon RDS, which ensures that all data models are stored securely and backed up automatically.

9. Data View

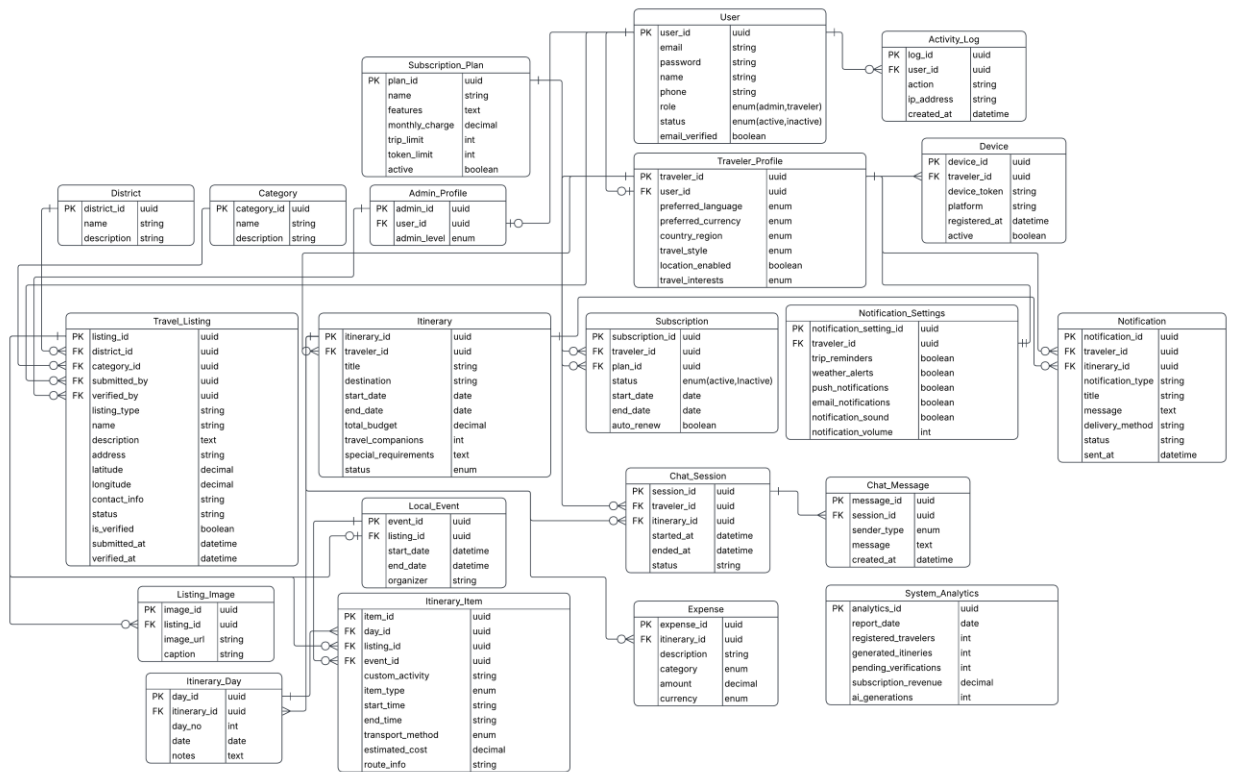


Figure 17: ER Diagram

10. Size and Performance

10.1 Web Application Loading Time

Developers are encouraged to make the size of a web page less than 10MB. Hence, it will only take 1 second to load the webpage under 1MBps connection. For the worst-case scenario, the web page will load within 8 seconds under 128KB/s connection.

10.2 Mobile App Loading Time

Android app do not need to load any GUI from the server. Android app only need to communicate with the REST API. Most of the time, the data sent to and received from the REST server is less than 500KB. Due to this Average response time of the Android app will be less than 500ms under 1MBps connection

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

10.3 AI Itinerary Generation Requests Per Second

System uses Gemini API service together with Google maps API, Weather API to generate personalized travel itineraries. the backend should process up to 20 AI itinerary generation requests per second. If the number of requests exceeds this limit, incoming requests should be queued and processed as system resources become available.

10.4 Number of Database Requests Per Second

The system uses PostgreSQL and this should support atleast 300 database requests per second while maintaining acceptable response times for both web and mobile users.

10.5 Number of Simultaneous Users

The web app will be hosted on Amazon Web Server (AWS) free tier. This can handle 20,000 concurrent users. However, this limitation will only be valid for the web app. There are no limitations for android app installation. But there is a limitation to the requests done by both web and mobile apps. The maximum request for REST API by both android and web app should not exceed 1000.

10.6 System Up Time

System should have at least 99% of the uptime. The users should be informed about the maintenance at least one week before. The maintenance period should be less than 12 hrs.

10.7 Resource Utilization

System should host on a server which has at least 4 cores, 8GB memory, 2.5GHz clock speed, 250GB HDD. Moreover the server should have at least 2GB bandwidth.

11. Quality

11.1 Scalability

Separating the main backend (NestJS) from the AI processing service (Python/FastAPI) significantly increases scalability. If a large crowd is generating AI itineraries simultaneously, the heavy processing load is isolated. This ensures the high traffic will not crash the main application, allowing standard features like user authentication and profile management to run smoothly.

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

11.2 Maintainability

The strict separation of Presentation, Application, and Data layers, alongside a consistent, modular folder structure, improves the maintainability of the system. Utilizing proper REST API documentation, simple element naming conventions, and standard coding practices ensures that any developer can easily understand and update the codebase.

11.3 Extensibility

The loosely coupled microservice architecture and the modular Agentic AI system make the platform highly extensible. Because the AI is divided into specific agents (e.g., Weather Agent, Budget Agent, Policy Agent), developers can easily introduce new agents or external APIs in the future without having to rewrite or disrupt the existing core system.

11.4 Reliability

The system ensures high reliability by grounding the AI recommendations strictly on curated, verified data stored in the PostgreSQL database via the Admin Panel. This eliminates AI hallucinations and guarantees that users receive trustworthy travel information. Additionally, fallback mechanisms in the API layer prevent external service failures (like a weather API timeout) from crashing the whole itinerary generation process.

11.5 Portability & Compatibility

The platform is built using platform-independent and cross-platform stacks (Flutter for mobile, Next.js for web). By containerizing the backend services and relying on standard REST HTTP communication, the system achieves high portability. It can be easily deployed and executed across different cloud environments (such as AWS) or operating systems without changing the core application logic.

11.6 Configurability

When system environments change or external services require updates, the application can be configured without difficulty. All third-party integrations (Gemini AI, Google Maps API, OpenWeather API) and database connection strings are isolated into centralized environment configuration files (e.g., .env). This allows maintenance teams to update credentials securely without touching the source code.

11.7 Attractiveness & Usability

User-friendly interfaces with modern, responsive designs provide high attractiveness to the application. Features such as an intuitive conversational AI chat interface, clear dynamic itinerary dashboards, and a consistent visual theme across both the web and mobile platforms ensure an engaging and seamless experience for travelers.

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

12. References

- [1] P. Kruchten, "The 4+1 View Model of Architecture," *IEEE Software*, vol. 12, no. 6, pp. 42–50, Nov. 1995.
- [2] Object Management Group, "OMG Unified Modeling Language (OMG UML) Specification," Version 2.5.1, Dec. 2017. [Online]. Available: <https://www.omg.org/spec/UML/2.5.1/>
- [3] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, UK: John Wiley & Sons, 2009.
- [4] L. Xiang and R. Chen, "Agentic AI systems for dynamic travel itinerary planning and personalized tourism," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 4, pp. 1120–1132, Apr. 2024.
- [5] Next.js, "Next.js Documentation," *Vercel*, 2026. [Online]. Available: <https://nextjs.org/docs>. [Accessed: Aug. 9, 2026].
- [6] Flutter, "Flutter Architectural Overview," *Google*, 2026. [Online]. Available: <https://docs.flutter.dev/resources/architectural-overview>. [Accessed: Aug. 9, 2026].
- [7] NestJS, "NestJS Documentation - A Progressive Node.js Framework," 2026. [Online]. Available: <https://docs.nestjs.com/>. [Accessed: Aug. 9, 2026].
- [8] FastAPI, "FastAPI Web Framework Documentation," 2026. [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed: Aug. 9, 2026].
- [9] LangChain, "LangChain Python Documentation," 2026. [Online]. Available: <https://python.langchain.com/docs/>. [Accessed: Aug. 9, 2026].
- [10] Google, "Gemini API Overview," *Google AI for Developers*, 2026. [Online]. Available: <https://ai.google.dev/docs>. [Accessed: Aug. 9, 2026].
- [11] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2026. [Online]. Available: <https://www.postgresql.org/docs/16/>. [Accessed: Aug. 9, 2026].
- [12] Google, "Google Maps Platform Documentation," *Google Cloud*, 2026. [Online]. Available: <https://developers.google.com/maps/documentation>. [Accessed: Aug. 9, 2026].
- [13] OpenWeather, "OpenWeather API Documentation," 2026. [Online]. Available: <https://openweathermap.org/api>. [Accessed: Aug. 9, 2026].
- [14] Diagrams.net, "draw.io Diagramming Software," 2026. [Online]. Available: <https://app.diagrams.net/>. [Accessed: Aug. 9, 2026].
- [15] Lucid Software Inc., "Lucidchart Visual Collaboration," 2026. [Online]. Available:

Smart Tourism with Multi agent collaboration	Version: 1.0
Software Architecture Document	Date: 08/08/26

<https://www.lucidchart.com/>. [Accessed: Aug. 9, 2026].